

11

Reviewing

Performance

Scalability

Scalability is the ability of a system to handle an increasing amount of work or to expand its capacity to handle an increasing amount of work.

→ about if sys is working correctly?

→ about how fast the sys is?

Scalability is the ability of a system to handle an increasing amount of work or to expand its capacity to handle an increasing amount of work.

→ about when workload is high, but the sys works same. (Same perf time)

What is an arch. style?

An architectural style is a named way of organizing a software system. It gives rules about how parts connect and work together, and it helps achieve certain quality goals.

1) Monolithic Arch.

System where all parts of the app are built and deployed as one single unit.

→ one codebase, one deployment, one process.



- Pros**
- Easy to develop
 - Easy to deploy
 - Simple arch.
- Disadv**
- High coupling (everything is coupled)
 - Hard to test
 - Scale is problematic



When is it Suitable?

- Small systems
- Simple applications
- Early prototypes
- Simple projects

- Important Exam Idea**
- Monolithic systems are easy to develop but hard to maintain.
 - As system grows, it becomes harder to manage.

Important Exam Idea

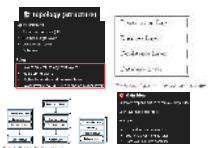
- Monolithic systems are easy to develop but hard to maintain.
- As system grows, it becomes harder to manage.



2) Layered Arch.

is a sys divided into sep. layers where each layer has a specific purpose and communicates only with its neighboring layers.

→ each layer builds on top of the one below it.



- Pros**
- Abstraction
 - Reuse
 - Clear separation of concerns
 - Easy to maintain
 - Scalability
 - Testability
 - Standardized layer interfaces for 3rd and 4th parties
- Disadv**
- Performance overhead
 - May not suit modern infrastructures (e.g. microservices)



When is it Suitable?

- Web applications
- Business applications
- Systems with clear separation of responsibilities

Important Exam Idea

- Layered architecture is a good way to organize a system.
- It allows for better testing and maintenance.



3) Client-server

is a sys where clients req. services and server provides services. It manages data over a network.

→ can be called 2-tier.

→ client and server are sep. and often on diff. machines.



- Pros**
- Centralized data management
 - Server can be bottleneck
 - If server crashes, users can't access data
 - Network latency affects perf.
 - Reg. network 3rd party
- Disadv**
- Server can be bottleneck
 - If server crashes, users can't access data
 - Network latency affects perf.
 - Reg. network 3rd party

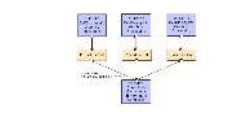


When is it Suitable?

- Web applications
- Business applications
- Systems with clear separation of responsibilities

Important Exam Idea

- Client-server architecture is a good way to organize a system.
- It allows for better testing and maintenance.



4) Pipe and Filter

is a style where data flows through a series of independent components (filters), connected by pipes.

→ each filter receives input data, processes it, and produces output data.



- Pros**
- High modularity
 - Easy to add new filters
 - Good for data flow
 - Server can be bottleneck
- Disadv**
- Not suitable with interactive apps
 - Not good for complex algorithms
 - Network latency affects perf.
 - Reg. network 3rd party

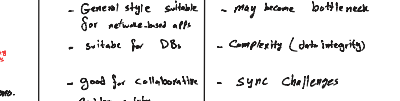
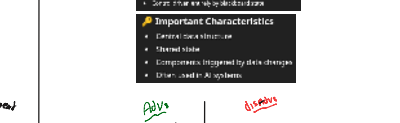
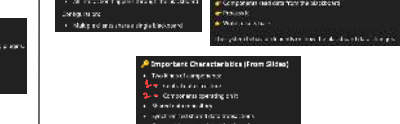
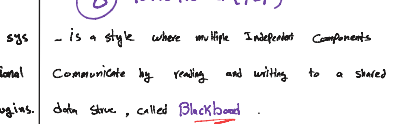
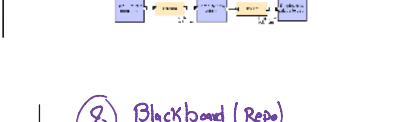
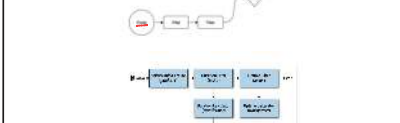


When is it Suitable?

- Web applications
- Business applications
- Systems with clear separation of responsibilities

Important Exam Idea

- Pipe and filter architecture is a good way to organize a system.
- It allows for better testing and maintenance.



Layered

- state - logic - display

- Model - view - Controller MVC

Definition

Layered patterns are architectural patterns where the system is divided into layers, each layer has a specific responsibility.

The idea is to separate parts of the system that change at different rates.

Your slide says:

"Separate elements with different rate of change"

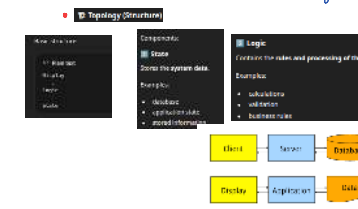
Meaning:

Some parts of a system change often (like UI), while others change rarely (like database).

Layered patterns separate these parts to make the system easier to maintain.

State-logic

→ separates the sys into state, logic, display



Main Idea

Different parts of software change at different speeds.

Example:

Component	Change Frequency
Display	changes very often
Logic	changes sometimes
State	changes rarely

So the architecture separates them.

Advantages

- Modularity
- Reusability
- Maintainability
- Scalability

Disadvantages

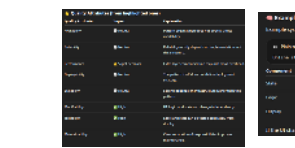
- Performance
- Complexity
- Testing
- Deployment

Suitable Applications

- UI changes frequently
- Logic should remain stable
- data must be protected

Examples:

- desktop applications
- early GUI systems
- interactive software



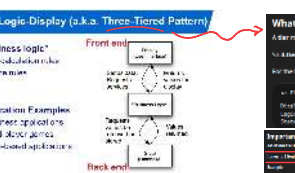
Additional notes

State-logic-Display (a.k.a. Three-Tiered Pattern)

- "Business logic"
- The calculation rules
- The data processing

Application Examples:

- Business applications
- Multi-tiered systems
- Distributed applications



State-logic-Display (a.k.a. Three-Tiered Pattern)

Fundamental rule:

- No direct communication between Display and State

Display, Logic and State:

- are developed and maintained independently
- most often on separate platforms
- often using different technologies



Definition

Component patterns focus on how independent components or services communicate with each other.

These patterns are especially important in:

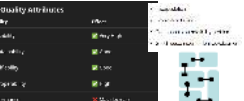
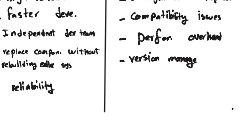
- distributed systems
- microservices
- service-based architectures

5) Component-Based Arch

is a style where a sys is built from independent reusable components that communicate through interfaces.

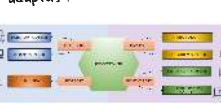


- Pros**
- high reusability
 - faster dev.
 - independent team
 - replace comp. without affecting other
 - testability
- Disadv**
- Integration Complex
 - Compatibility issues
 - Perform overhead
 - version manage



6) Hexagonal (Ports & Adapters)

is a style where the core business logic is placed in the center, and all external sys (UI, db, API, ...) connect to it through ports and adapters.



When is it Suitable?

- Web applications
- Business applications
- Systems with clear separation of responsibilities

Important Exam Idea

- Hexagonal architecture is a good way to organize a system.
- It allows for better testing and maintenance.



7) Plugin or microkernel

is a style where a small core sys provides basic func., and additional features are added as independent plugins.

→ e.g. browser, game app, editor, etc.

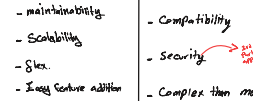


When is it Suitable?

- Web applications
- Business applications
- Systems with clear separation of responsibilities

Important Exam Idea

- Plugin architecture is a good way to organize a system.
- It allows for better testing and maintenance.



8) Blackboard (Repo)

is a style where multiple independent components communicate by reading and writing to a shared data store, called Blackboard.

→ the blackboard contains like app, p, o, etc.

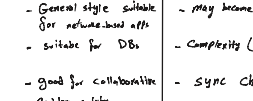


When is it Suitable?

- Web applications
- Business applications
- Systems with clear separation of responsibilities

Important Exam Idea

- Blackboard architecture is a good way to organize a system.
- It allows for better testing and maintenance.



Distributed System

- A system with multiple components located on *different machines* that communicate and coordinate actions in order to *appear as a single coherent system* to the end-user.

What are the benefits?

- Improved *reliability*
- Improved *scalability*
- Improved *latency*

What are the drawbacks?

- Increased *complexity*
- Increased *attack surface*
- Increased *latency*
- Introduce *consistency* problems

1 Service oriented Arch

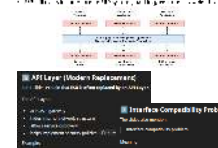
S.O Arch is the composition of software components (services) to form a complete sys.



Topology (Structure)



- Advantages:
 - Loose coupling between services
 - Reusability of services
 - Good interoperability
 - Integrates legacy sys.
 - scalable for large enterprise
 - standard interfaces & protocols
- Disadvantages:
 - ESB problems: central bottleneck
 - hard to scale
 - heavy synchronous
 - Complex setup
 - Business logic moving into ESB
 - Other issues:
 - Shared DB makes service hard to manage.
 - Failure of one service may affect workflow.

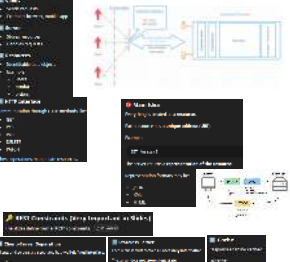


2 REST

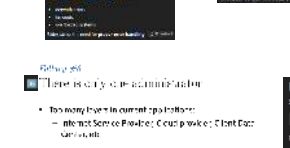
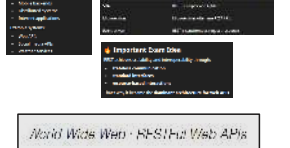
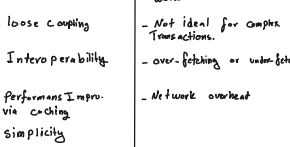
- is an arch. style for distributed sys defining certain constraints that provide efficiency, scal. and other desirable attr.



Topology (Structure)



- Advantages:
 - Scalable
 - Loose coupling
 - Interoperability
 - Performs improv. via caching
 - Simplicity
- Disadvantages:
 - Statelessness on increase work.
 - Not ideal for complex transactions.
 - over-fetching or under-fetch.
 - Network overhead



3 microservices Arch.

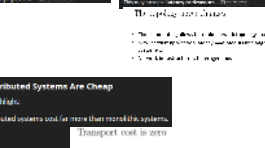
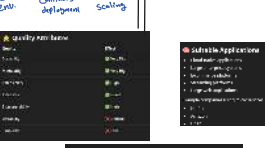
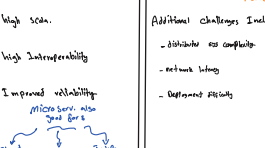
- is an evolution of service-oriented Arch (SOA)



Topology (Structure)



- Advantages:
 - High modularity
 - High extensibility
 - High scalability
 - High interoperability
 - Improved reliability
- Disadvantages:
 - Microservices are not just APIs
 - Additional challenges include:
 - distributed sys complexity
 - network latency
 - Deployment complexity



4 Distributed Monolith (Anti-Pattern)

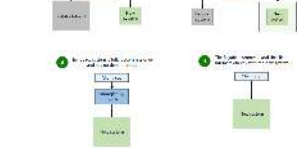
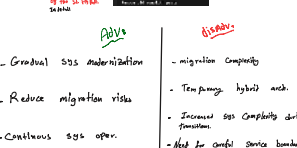
- The essence of the distributed monolith is lack of isolation between services.



Topology (Structure)



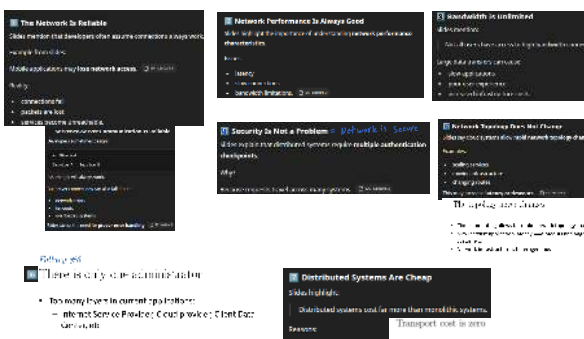
- Advantages:
 - Quality Attributes
- Disadvantages:
 - When Does This Happen?



- Advantages:
 - Gradual sys modernization
 - Reduce migration risks
 - Continuous sys open.
 - Better scale.
 - Improved modularity.
- Disadvantages:
 - migration complexity
 - Temporary hybrid arch.
 - Increased sys complexity during transition.
 - Not for central service bundles

Slays last part

Fallacies of Distributed Computing



B - Arch. by Implication

Definition

The architecture by implication and system history refers to a system that no explicitly defined architecture, and therefore usually the architecture and design responsibility for the development lies with the system owner.

Architecture is not explicitly defined or documented. Decisions are made to solve issues individually and the system grows. (B.1) (B.2)

Common features:

- Incremental growing architecture. Decisions only "We will implement it now."
- Architecture is developed incrementally rather than being designed.

Topology (Structural)

Architecture by implication is a top-down approach. It is characterized by a lack of formal structure and a focus on incremental growth. The system evolves over time, often driven by immediate needs and constraints.

Main Problem

Because architecture is not defined:

- development team build decisions
- development team build decisions
- the system eventually becomes hard to maintain

Over time the system may become a Ball of Mud.

There is no clear architectural design or documentation

Example

Development team build a web system.

Instead of defining architecture:

- Developers just start coding features.
- Later the system evolves.
- Incremental growing architecture.
- Common module boundaries.

Key Features:

- Incremental growth
- Common module boundaries
- Developers just start coding features
- Later the system evolves
- Incremental growing architecture
- Common module boundaries

Quality Attributes Impact

Quality Attributes: Reliability, Availability, Performance, Security, Maintainability, Portability, Scalability, Interoperability, Usability, Compliance.

Impact:

- Reliability: **Reliability is often a secondary concern.**
- Availability: **Availability is often a secondary concern.**
- Performance: **Performance is often a secondary concern.**
- Security: **Security is often a secondary concern.**
- Maintainability: **Maintainability is often a secondary concern.**
- Portability: **Portability is often a secondary concern.**
- Scalability: **Scalability is often a secondary concern.**
- Interoperability: **Interoperability is often a secondary concern.**
- Usability: **Usability is often a secondary concern.**
- Compliance: **Compliance is often a secondary concern.**

Suggested Solution (from lecture)

The solution is explicit architectural design.

Recommended steps:

- Define architecture early.
- Document architectural decisions.
- Use architectural patterns.
- Review architecture regularly.

Architecture should be defined incrementally, not document architecture.

Real-World Example

Startups sometimes fall into this anti-pattern when they prioritize "Ship Features Fast!"

Without designing architecture.

Later, when the system grows, they must refactor the entire system.

Real-World Example

Startups sometimes fall into this anti-pattern when they prioritize "Ship Features Fast!"

Without designing architecture.

Later, when the system grows, they must refactor the entire system.

Problems

- Architecture is often a secondary concern.
- Availability is often a secondary concern.
- Performance is often a secondary concern.
- Security is often a secondary concern.
- Maintainability is often a secondary concern.
- Portability is often a secondary concern.
- Scalability is often a secondary concern.
- Interoperability is often a secondary concern.
- Usability is often a secondary concern.
- Compliance is often a secondary concern.

Solutions

- As organized approach to systems architecture definition, that relies on multiple views of the system.

Definition

The architecture by implication and system history refers to a system that no explicitly defined architecture, and therefore usually the architecture and design responsibility for the development lies with the system owner.

Architecture is not explicitly defined or documented. Decisions are made to solve issues individually and the system grows. (B.1) (B.2)

Common features:

- Incremental growing architecture. Decisions only "We will implement it now."
- Architecture is developed incrementally rather than being designed.

Main Problem

Because architecture is not defined:

- development team build decisions
- development team build decisions
- the system eventually becomes hard to maintain

Over time the system may become a Ball of Mud.

Negative Effects

Because architecture is not defined:

- development team build decisions
- development team build decisions
- the system eventually becomes hard to maintain

Over time the system may become a Ball of Mud.

Quality Attributes Impact

Quality Attributes: Reliability, Availability, Performance, Security, Maintainability, Portability, Scalability, Interoperability, Usability, Compliance.

Impact:

- Reliability: **Reliability is often a secondary concern.**
- Availability: **Availability is often a secondary concern.**
- Performance: **Performance is often a secondary concern.**
- Security: **Security is often a secondary concern.**
- Maintainability: **Maintainability is often a secondary concern.**
- Portability: **Portability is often a secondary concern.**
- Scalability: **Scalability is often a secondary concern.**
- Interoperability: **Interoperability is often a secondary concern.**
- Usability: **Usability is often a secondary concern.**
- Compliance: **Compliance is often a secondary concern.**

Real-World Example

Startups sometimes fall into this anti-pattern when they prioritize "Ship Features Fast!"

Without designing architecture.

Later, when the system grows, they must refactor the entire system.

Problems

- Architecture is often a secondary concern.
- Availability is often a secondary concern.
- Performance is often a secondary concern.
- Security is often a secondary concern.
- Maintainability is often a secondary concern.
- Portability is often a secondary concern.
- Scalability is often a secondary concern.
- Interoperability is often a secondary concern.
- Usability is often a secondary concern.
- Compliance is often a secondary concern.

Solutions

- As organized approach to systems architecture definition, that relies on multiple views of the system.

Step	Description
Identify bounded contexts	determine
Create service boundaries	define se
Extract services	move fu
Redirect requests	gradual
Refactor monolith	remove

- ② Component :
- Interoperability
 - Directory Pattern :
- ③ Events :
- Event Monitor
 - Publish - Subscriber
 - Messaging bridge
 - half sync / half async
- ④ Composition :
- Scatter - Gather
 - Load Balancing
 - Orchestration
 - Choreography
 - Saga pattern
 - Map - Reduce

⑤ Layered :

Main Idea Instead of mixing everything together	Why Layered Patterns Are Used They improve: Quality Attribute Why
Plan text UI + Data + Data to user phone Layered pattern separates them	Maintainability easier to modify developer
Plan text UI + Data + Data to user phone Layered pattern separates them	Modifiability UI can change without changing logic
Plan text UI + Data + Data to user phone Layered pattern separates them	Reusability logic can be reused with different UI
Plan text UI + Data + Data to user phone Layered pattern separates them	Testability layers can be tested separately

- 3 Components:
- Display
2. Model - View - Controller
- separates an app into 3 components
- Model
 - View
 - Controller

Display [renders how information is presented to users] User Interface UI UI page UI layout	Model-View-Controller (MVC) Basic structure: Plan text UI + Data + Data to user phone Layered pattern separates them Or conceptually: Plan text UI + Data + Data to user phone Layered pattern separates them	Components Explained Model Responsible for: application data business logic system state Examples: product database user accounts settings business rules	View Responsible for: displaying information to users Examples: web pages mobile screens dashboards [The view reads data from the model.]	Controller Responsible for: handling user input operating the model updating the view to display Example: web page mobile screen dashboard
---	--	---	---	---

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Key Concepts pattern focuses on: separation of concerns isolating system data isolating presentation layer controlling how logic interacts with state

Quality Attributes	Impact
Scalability	poor
Maintainability	poor
Testability	reduced

Migration Strategy	Description
Strangler Fig Pattern	gradually replace monolith with services

Challenges of Distributed Systems

Challenge	Explanation
network latency	slower communication
fault tolerance	systems must handle failures
data consistency	difficult across services
monitoring	harder to observe system state

Lecture 3 Key Takeaways

Key Idea	Explanation
Modern systems are distributed	
Microservices enable scalability	
Poor microservices design leads to distributed monolith	
Distributed systems introduce complexity	
Migration from monolith should be gradual	

Lecture 4 — Architectural Patterns & Anti-Patterns

Architectural Patterns	Pattern Group	Pattern Group
Layered Patterns	Component Patterns	Event Patterns
Scatter-Gather	Orchestration	Choreography
Load Balancing	Saga Pattern	Map-Reduce

Pattern Groups in the Lecture	Pattern Group	Pattern Group
Layered Patterns	Component Patterns	Event Patterns
Scatter-Gather	Orchestration	Choreography
Load Balancing	Saga Pattern	Map-Reduce

Model-View-Controller (MVC)	Model	View	Controller
Model	application data and business logic	user interfaces	manages interaction between model and view
View			
Controller			

Component Patterns	Interoperability Pattern
Interoperability Pattern	
Interoperability Pattern	
Interoperability Pattern	
Interoperability Pattern	

Event Patterns	Event Monitor
Event Monitor	
Event Monitor	
Event Monitor	
Event Monitor	

Composition Patterns	Scatter-Gather
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	

Event Patterns	Scatter-Gather
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	

Composition Patterns	Scatter-Gather
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	

Event Patterns	Scatter-Gather
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	

Composition Patterns	Scatter-Gather
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	

Event Patterns	Scatter-Gather
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	

Composition Patterns	Scatter-Gather
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	

Event Patterns	Scatter-Gather
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	

Composition Patterns	Scatter-Gather
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	
Scatter-Gather	

Lecture 4 Key Takeaways

Key Idea	Explanation
Architectural patterns provide reusable architecture solutions	MVC, Saga, MapReduce
Patterns are grouped by problem type	layered, component, event, composition
Anti-patterns represent bad design practices	should be avoided
Good architecture improves quality attributes	scalability, maintainability

Summary of Sum.

Ultra-Short Memory Version (Exam Recall)

Lecture	5-Word Memory Cue
Lecture 1	Why architecture exists
Lecture 2	Ways to structure systems
Lecture 3	How distributed systems work
Lecture 4	Solutions and mistakes in architecture

Concept Flow of the Course

Lecture	5-Word Memory Cue
Lecture 1	Architecture fundamentals
Lecture 2	Architecture structures (styles)
Lecture 3	Modern distributed architectures
Lecture 4	Modern distributed architectures

What This Helps With In the

Question	Answer
What is software architecture?	
What architecture structures exist?	
How are modern distributed systems built?	
How do we solve architectural problems?	

② Component :

Main Idea Component patterns help solve problems like: how components find each other how systems communicate across technologies how distributed services coordinate	Patterns in the Component Group From the lecture, this group includes: Interoperability Pattern Directory Pattern
--	---

Patterns in the Component Group

From the lecture, this group includes: Interoperability Pattern Directory Pattern

- written in different languages
- running on different servers
- using different technologies

Component patterns **enable interoperability** between them.

[illegible]

Example Technologies

- IBM J9RE
- Parbo
- Sub4MQ

These provide platform agnostic communications

Exploration
communication layers required
network communication overhead
standards must be maintained

- **Real-World Example**
- Healthcare systems often integrate:
 - hospital software
 - insurance systems
 - pharmacy systems
- All using **Interoperability standards**.

... a good person.

- Definition**
 The **Directory Pattern** allows components or services in a distributed system to find each other **dynamically** without knowing their exact location.
 From that follows:
 It provides **location transparency**, meaning components can discover services without knowing their physical addresses.
- In simple words:
 Instead of hardcoding service addresses, systems ask a central directory where the service is located.

The diagram illustrates the Service Registry (Directory) and its interactions with Clients and Service Instances.

Service Registry (Directory):

- Plan view:**
 - Client:**
 - Service Registry (Directory)
 - Service Instance:**
- Example flow:**
 - Service registers itself in the **directory**
 - Client queries the **directory**
 - Client requests the **service address**
 - Client communicates with the **service instance**

Client:

- Application generating a request
- Language
- Web application
- Microservice
- API gateway

Service Instance:

- Application generating a request
- Example:
 - Application server
 - Database server
 - Web server

Client-Side Discovery

The client directly queries the service registry.

```
graph LR
    Client --> Registry
    Registry --> Service
```

Process:

- 1. Client asks registry for service location
- 2. Registry returns service instance
- 3. Client communicates directly with the service

Example technologies:

- Netflix Eureka
- Consul

Server-Side Discovery

The client sends requests to a **load balancer or gateway**, which handles discovery.

```
graph LR; Client --> LB[Load Balancer]; LB --> Service; Client -.-> Service
```

The diagram illustrates the flow of a request in a server-side discovery architecture. A red arrow points from 'Client' to 'Load Balancer', and another red arrow points from 'Load Balancer' to 'Service'. A dashed red arrow points directly from 'Client' to 'Service'. A red circle with a red arrow indicates a loop or feedback mechanism from the 'Service' back to the 'Load Balancer'.

Process:

1. Client contacts load balancer
2. Load balancer queries registry
3. Load balancer forwards request

Example technologies:

- Kubernetes service discovery
- AWS load balancer



- Client sends request to a fixed endpoint, usually a load balancer or API gateway.
- The load balancer queries the service registry to pick a healthy service instance.
- Load balancer forwards the request to that service instance.
- Client *only* interacts with the load balancer.

```
instead of:

...
Client : Service IP address

You have:

...
Client : Directory : Service

This allows services to move or scale without breaking the system.
```

Advantages	
Advantage	Explanation
Location transparency	clients don't need to know service location
Dynamic service discovery	services can move at scale
Flexibility	easier service management
Improved scalability	services can be replicated

Disadvantages	
Disadvantage	Explanation
Service infrastructure	registry server required
Complexity: single point of failure	if registry fails
Additional network communication	looking up endpoint

Quality dimension	Impact	Explanation
Availability		

- microservice architecture
- cloud systems
- distributed applications
- service-oriented architectures

Example: DMG (Domain Name System)

Instead of using server IP:

```
++ Hostname:
192.168.79.14
```

OR USE:

```
++ Hostname:
google.com
```

IPV6 is necessary that maps names to locations

discover services in distributed environments.

Objects react to events instead of calling each other

nts and react when they occur.

```
instead of direct communication:

// Plain text
Service A - Service B

Event systems work like:

// Plain text
Event Happens - Components react

Example:

// Plain text
Order Placed Event
Payment Service
Shipping Service
Notification Service

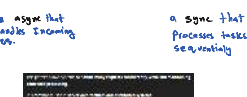
Each component subscribes to events and responds when needed.
```

an event-based arch where publishers
msgs (event) and subs receive them,
knowing each other directly.

is used to connect two diff. msgg.
buses. or msgg. buses that are not
directly compatible.

- separates a sig to 2 parts

a sig that a sig th



Add a layer hiding asynchronous interactions behind a



Downloaded from <http://ajphaphysocpharm.sagepub.com/>

He has been a frequent speaker at national and international conferences and has published numerous articles in the field of international law.

Lecture	Core Focus	Main Question Answered	Key Concepts	Architecture Level	Example Topics	One Key Takeaway
Lecture 1	Fundamentals of Software Architecture	Why architecture matters and what it consists of	components, connectors, quality attributes, stakeholders, architectural decisions	Conceptual foundation	quality attributes, architectural views, trade-offs	Architects define the blueprint for the system, like a city planner.
Lecture 2	Architectural Styles	What structural patterns can organize a system	components, connectors, topology, constraints	Structural system design	layered, client-server, pipe-filter, event-driven, P2P	Architectural style is like an organizational structure, defining how parts interact.
Lecture 3	Distributed Architectures	How modern large systems are built and deployed	services, distributed systems, microservices, REST, system migration	System architecture evolution	SOA, REST, microservices, distributed monolith, migration patterns	Modern systems are often distributed, moving from monoliths to microservices.
Lecture 4	Architectural Patterns & Anti-Patterns	How to solve architectural problems and avoid bad designs	reusable patterns, event systems, service coordination, architecture mistakes	Architectural problem-solving	MVC, Saga, MapReduce, orchestration, anti-patterns	Architects use patterns to solve common problems and avoid known pitfalls.

Definitions (Very Common Questions)

Question	Expected Answer
What is Software Architecture?	The high-level structure of a software system , including components, connectors, and architectural decisions that achieve system goals effectively.
What is an Architectural Style?	A family of systems defined by components, connectors, languages, and constraints that achieve those goals.
What is an Architectural Pattern?	A reusable solution to a recurring architectural problem in a specific context.
Difference between architectural style and pattern	Style defines system structure , pattern provides solution to architectural problem .
Difference between architectural patterns and design pattern	Architectural patterns apply to system architecture , design patterns apply to class-level code .

Attribute	Meaning
Availability	system operational anytime
Reliability	system works without failure
Performance	response time and throughput
Scalability	ability to handle increased workload
Deployability	ease of deploying system
Modifiability	ease of changing functionality
Testability	ease of testing system components
Maintainability	ease of maintaining system

- Monolithic
- Layered
- Client-Server
- Pipe & Filter
- Component-Based
- Event-Driven
- Peer-to-Peer

4 Microservices

Question

What is Microservices?

Main principle

Main advantages

Main disadvantages

Question	Answer
What is REST?	architectural style for stateless web services using HTTP .
REST operations	GET, POST, PUT, DELETE
REST principle	stateless communication

6 Distributed System Questions

Question	Answer
What is a distributed monolith?	system appears distributed but services are tightly coupled like a monolith.
Why distributed systems are complex	network latency, failures, consistency challenges
Migration pattern from monolith	Strangler Fig pattern

Feature	Monolith	Microservices
Deployment	single unit	independent services
Scalability	limited	high
Complexity	low	high
Maintenance	difficult as system grows	easier modular updates

Client-Server vs Peer-to-Peer		
Feature	Client-Server	Peer-to-Peer
Control	centralized server	decentralized
Scalability	moderate	high
Fault tolerance	server failure critical	more resilient

Pattern	Key
MVC	separation of concerns
Directory Pattern	service discovery
Publish-Subscribe	pub-sub
Scatter-Gather	distributed computing
Load Balancing	distributed load
Orchestration	centralized control
Choreography	service-to-service
Saga	distributed transactions
MapReduce	distributed data processing

Orchestration vs Choreography	
Feature	Orchestration
Control	centralized
Workflow	controlled by orchestrator
Scalability	moderate
Monitoring	easier

Anti-pattern	Meaning
Stovepipe System	ad hoc integrations between systems
Stovepipe Enterprise	isolated enterprise systems
Autogenerated Stovepipe	legacy monoliths wrapped as services
Ball of Mud	chaotic architecture
Architecture by Implication	architecture not explicitly defined
Rainbow the Wheel	building winning solutions
Vendor Lock-in	dependence on specific vendor technology

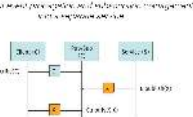
Group	Purpose	Patterns
Layered	separate responsibilities	State Logic Display, MVC
Component	communication between components	Interoperability, Directory
Event	event-based communication	Twelve Monitors, Pub/Sub, Messaging Bridge
Composition	coordinate multiple services	Scatter-Gather, Load Balancing, Saga

an event-based arch where publishers
msgs (event) and subs receive them,
knowing each other directly.

is used to connect two diff. msgg.
buses. or msgg. buses that are not
directly compatible.

- separates a sigc to 2 parts

a sigc that a sigc th



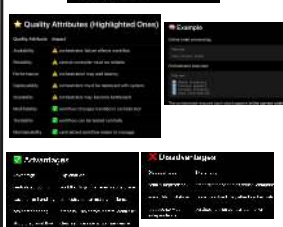
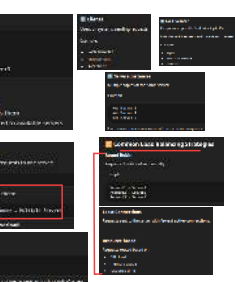
1. **Introduction**
 2. **Background**
 3. **Methodology**
 4. **Results**
 5. **Conclusion**
 6. **References**
 7. **Appendix**
 8. **Index**
 9. **Table of Contents**
 10. **Figure 1**
 11. **Figure 2**
 12. **Figure 3**
 13. **Figure 4**
 14. **Figure 5**
 15. **Figure 6**
 16. **Figure 7**
 17. **Figure 8**
 18. **Figure 9**
 19. **Figure 10**
 20. **Figure 11**
 21. **Figure 12**
 22. **Figure 13**
 23. **Figure 14**
 24. **Figure 15**
 25. **Figure 16**
 26. **Figure 17**
 27. **Figure 18**
 28. **Figure 19**
 29. **Figure 20**
 30. **Figure 21**
 31. **Figure 22**
 32. **Figure 23**
 33. **Figure 24**
 34. **Figure 25**
 35. **Figure 26**
 36. **Figure 27**
 37. **Figure 28**
 38. **Figure 29**
 39. **Figure 30**
 40. **Figure 31**
 41. **Figure 32**
 42. **Figure 33**
 43. **Figure 34**
 44. **Figure 35**
 45. **Figure 36**
 46. **Figure 37**
 47. **Figure 38**
 48. **Figure 39**
 49. **Figure 40**
 50. **Figure 41**
 51. **Figure 42**
 52. **Figure 43**
 53. **Figure 44**
 54. **Figure 45**
 55. **Figure 46**
 56. **Figure 47**
 57. **Figure 48**
 58. **Figure 49**
 59. **Figure 50**
 60. **Figure 51**
 61. **Figure 52**
 62. **Figure 53**
 63. **Figure 54**
 64. **Figure 55**
 65. **Figure 56**
 66. **Figure 57**
 67. **Figure 58**
 68. **Figure 59**
 69. **Figure 60**
 70. **Figure 61**
 71. **Figure 62**
 72. **Figure 63**
 73. **Figure 64**
 74. **Figure 65**
 75. **Figure 66**
 76. **Figure 67**
 77. **Figure 68**
 78. **Figure 69**
 79. **Figure 70**
 80. **Figure 71**
 81. **Figure 72**
 82. **Figure 73**
 83. **Figure 74**
 84. **Figure 75**
 85. **Figure 76**
 86. **Figure 77**
 87. **Figure 78**
 88. **Figure 79**
 89. **Figure 80**
 90. **Figure 81**
 91. **Figure 82**
 92. **Figure 83**
 93. **Figure 84**
 94. **Figure 85**
 95. **Figure 86**
 96. **Figure 87**
 97. **Figure 88**
 98. **Figure 89**
 99. **Figure 90**
 100. **Figure 91**
 101. **Figure 92**
 102. **Figure 93**
 103. **Figure 94**
 104. **Figure 95**
 105. **Figure 96**
 106. **Figure 97**
 107. **Figure 98**
 108. **Figure 99**
 109. **Figure 100**
 110. **Figure 101**
 111. **Figure 102**
 112. **Figure 103**
 113. **Figure 104**
 114. **Figure 105**
 115. **Figure 106**
 116. **Figure 107**
 117. **Figure 108**
 118. **Figure 109**
 119. **Figure 110**
 120. **Figure 111**
 121. **Figure 112**
 122. **Figure 113**
 123. **Figure 114**
 124. **Figure 115**
 125. **Figure 116**
 126. **Figure 117**
 127. **Figure 118**
 128. **Figure 119**
 129. **Figure 120**
 130. **Figure 121**
 131. **Figure 122**
 132. **Figure 123**
 133. **Figure 124**
 134. **Figure 125**
 135. **Figure 126**
 136. **Figure 127**
 137. **Figure 128**
 138. **Figure 129**
 139. **Figure 130**
 140. **Figure 131**
 141. **Figure 132**
 142. **Figure 133**
 143. **Figure 134**
 144. **Figure 135**
 145. **Figure 136**
 146. **Figure 137**
 147. **Figure 138**
 148. **Figure 139**
 149. **Figure 140**
 150. **Figure 141**
 151. **Figure 142**
 152. **Figure 143**
 153. **Figure 144**
 154. **Figure 145**
 155. **Figure 146**
 156. **Figure 147**
 157. **Figure 148**
 158. **Figure 149**
 159. **Figure 150**
 160. **Figure 151**
 161. **Figure 152**
 162. **Figure 153**
 163. **Figure 154**
 164. **Figure 155**
 165. **Figure 156**
 166. **Figure 157**
 167. **Figure 158**
 168. **Figure 159**
 169. **Figure 160**
 170. **Figure 161**
 171. **Figure 162**
 172. **Figure 163**
 173. **Figure 164**
 174. **Figure 165**
 175. **Figure 166**
 176. **Figure 167**
 177. **Figure 168**
 178. **Figure 169**
 179. **Figure 170**
 180. **Figure 171**
 181. **Figure 172**
 182. **Figure 173**
 183. **Figure 174**
 184. **Figure 175**
 185. **Figure 176**
 186. **Figure 177**
 187. **Figure 178**
 188. **Figure 179**
 189. **Figure 180**
 190. **Figure 181**
 191. **Figure 182**
 192. **Figure 183**
 193. **Figure 184**
 194. **Figure 185**
 195. **Figure 186**
 196. **Figure 187**
 197. **Figure 188**
 198. **Figure 189**
 199. **Figure 190**
 200. **Figure 191**
 201. **Figure 192**
 202. **Figure 193**
 203. **Figure 194**
 204. **Figure 195**
 205. **Figure 196**
 206. **Figure 197**
 207. **Figure 198**
 208. **Figure 199**
 209. **Figure 200**
 210. **Figure 201**
 211. **Figure 202**
 212. **Figure 203**
 213. **Figure 204**
 214. **Figure 205**
 215. **Figure 206**
 216. **Figure 207**
 217. **Figure 208**

Our second test involves an analysis of the following:



or components are combined together to

attends client's incoming requests across multiple service instances or
 or a single backend service.



Definition

A distributed system is a collection of independent computers that appear to users as a single coherent system.

Characteristics

- Autonomy: Each node in the system operates independently.
- Scalability: The system can grow by adding more nodes.
- Reliability: The system can continue to operate even if some nodes fail.
- Transparency: Users are unaware of the underlying distributed nature of the system.

Components

- Nodes: Individual computers or devices in the system.
- Network: The communication infrastructure connecting the nodes.
- Middleware: Software that facilitates communication between nodes.
- Operating System: The software that manages the hardware resources of each node.

Applications

- Web Browsers: Accessing web pages from multiple servers.
- Database Systems: Storing and retrieving data across multiple nodes.
- File Systems: Managing files distributed across multiple storage devices.
- Real-time Systems: Coordinating multiple sensors and actuators.



Definitions

- **Network**: group of nodes (computers) connected in a certain structure
- **Network topology**: the way in which the nodes are connected to each other
- **Topology**: (Mathematics) the study of the properties of shapes and spaces that are independent of their size and shape

• **Network topology**: the way in which the nodes are connected to each other

The diagram illustrates a network topology. It features a central node labeled 'SERVER' connected to four peripheral nodes labeled 'CLIENT 1', 'CLIENT 2', 'CLIENT 3', and 'CLIENT 4'. The connections are represented by lines radiating from the central server to each client.

• **Network topology**: the way in which the nodes are connected to each other

Example:

The *Google* network consists of distributed IP addresses across multiple servers to facilitate large transactions into a sequence of smaller local transactions.

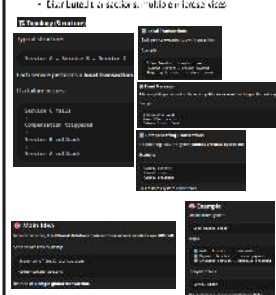
From left to right:

- Each server performs a local search for information at client that request for web transaction.
- It performs local calculations to transfer early process changes. (20-1000)
- It copies the data.

Instead of one big transaction across many servers, the system uses multiple smaller transactions.

Example:

```
graph LR; S1[Server 1  
Database] -- "Message/Query" --> S2[Server 2  
Database]; S2 -- "Message/Query" --> S3[Server 3  
Database];
```



The screenshot shows the 'New Project' dialog box in ArcGIS Pro. The 'Project Location' section is highlighted with a red box, showing the path 'C:\Users\jdoyle\OneDrive\Documents\GIS\Projects\GIS_101\GIS_101.aprx'. The 'Project Name' section is also highlighted with a red box, showing the name 'GIS_101.aprx'. The 'Project Type' section is highlighted with a red box, showing 'New Project'.

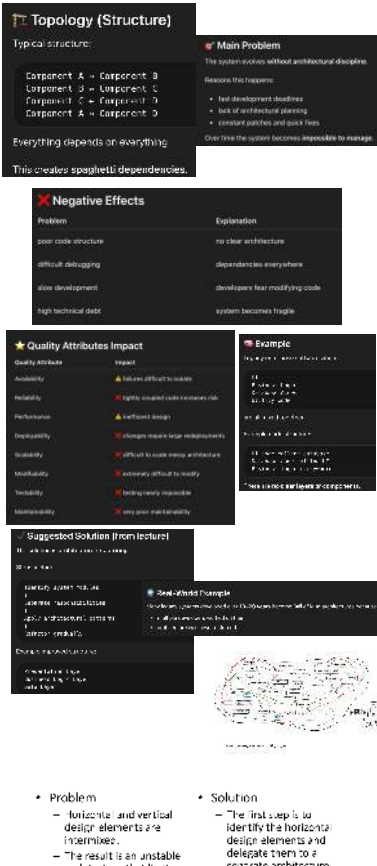
Scala patterns is used in:

- microservices architectures
- distributed transaction systems
- financial systems
- order processing systems

```

1 # The program reads a 10000 processing time logs dataset, using distributed execution
2 # completed
3
4 Command line:
5
6 # Reads the processing time logs dataset, using distributed execution
7 # Completed successfully
8
9 Time taken: 10.000 sec
10
11 Completed: 100% (10000 records)
12
13

```



- Then use the horizontal elements to capture the common interoperability functionality in the architecture.

6- Reinvent the wheel

7-vendor lock-in

Stakeholders in Software Architecture	
Stakeholder	Interest
Developers	System structure and implementation
Architects	Design decisions and trade-offs
Project managers	Cost and time factor
Users	Usability and performance
Operators	Reliability and maintenance

Architecture Trade-offs	
Architecture must balance competing qualities.	
Example	
Trade-off	Example
Performance vs. Maintainability	customized code harder to maintain
Flexibility vs. Efficiency	strong coupling may reduce flexibility
Scalability vs. Consistency	distributed systems increase complexity
Architecture often involves choosing the best compromise.	

Architectural Styles and Quality Attributes	
Quality Attribute	Impact of Architecture
Performance	depends on communication overhead
Scalability	distributed and stateless scale better
Reliability	redundant systems increase reliability
Maintainability	modular, extensible modules encourage maintainability
Modifiability	well-encapsulated system easier to extend

Client-Server Architecture	
Aspect	Description
Structure	clients request services from server
Components	clients, server
Advantages	centralized control
Disadvantages	server bottleneck
Quality Attributes	good maintainability but limited scalability
Example	
Client → Server → Database	

Plugin / Microkernel Architecture	
Aspect	Description
Structure	central system + plugins
Core	base system functionality
Plugins	extend system capabilities
Advantages	modifiability
Disadvantages	plugin management complexity
Quality Attributes	high modifiability
Example: IoT plugins	

Peer-to-Peer Architecture	
	Description
Structure	decentralized nodes
Components	each node acts as client and server
Advantages	high scalability and resilience
Disadvantages	security and coordination challenges
Quality Attributes	strong scalability

Microservices Architecture	
	Description
Structure	System composed of small independent services, each responsible for a specific business capability
Communication	REST APIs or messaging
Deployment	Independent deployment for each service
Advantages	
Modifiability	Modifiability
Scalability	services represent a specific domain
Reliability	services developed and deployed separately
Maintainability	minimal dependency between services
Disadvantages	
Complexity	Disadvantages
Deployment	distributed complexity
Scalability	difficult debugging
Reliability	network overhead
Quality Attributes	
Scalability	Scalability
Reliability	very high
Maintainability	high
Modifiability	very high
Complexity	difficult

Fallacies of Distributed Computing	
Common false assumptions when designing distributed systems.	
Fallacy	Reality
Network is reliable	networks fail
Latency is zero	network delays exist
Bandwidth is infinite	bandwidth limited
Network is secure	security risks exist
Topology doesn't change	distributed systems
One administrator	multiple control domains

	Explanation
	services communicate over networks
	independent service architecture
s	architecture must be planned carefully
	networking, failures, coordination
	use patterns like Strangler Fig

Behavioral Patterns	
Composite-Display Pattern	
description	- separate options into state, logic, and display - display → logic → state - separate parts that change in different ways
Pros / Disadvantages	
pros	flexible and elegant
cons	- additional complexity - increases class methods
Attributes	
scope	project
frequency	high
complexity	high
reusability	high

Discovery Pattern	Description
	- services dynamically discover each other - services registry or directory
Attributes	Description
Discovery	client queries registry
Discovery	load balancer performs lookup
Attributes	Impact
	high
	good

Architectural Anti-Patterns	
Architecture: Build and Maintain practices and business	Problem
Develop System	Uncoupled and modular responses
Develop Interface	Uncoupled external system organization
Implementer/Owner	Impery resources designed as functions
Control/Call of Interface	Shared system resources
Architecture by Location	Not planned self actions
Resource for Model	Self-organizing system
Control Interface	Control dependency for specific control

Impact of Anti-Patterns	
Quality Attribute	Block
Maintainability	Identify/Reused
Modifiability	Blocked
Flexibility	Blocked
Reliability	Unstable System

tion

Which Lecture
Lecture 1
Lecture 2
Lecture 3
Lecture 4

-Sentence Summary

Architecture defines the high-level structure of a system and determines how design decisions achieve quality attributes such as performance, scalability, and maintainability.

Architectural styles define common structural ways to organize system components and communication to achieve specific system qualities.

Modern software systems are distributed collections of services that communicate over networks, requiring careful design to manage scalability, reliability, and complexity.

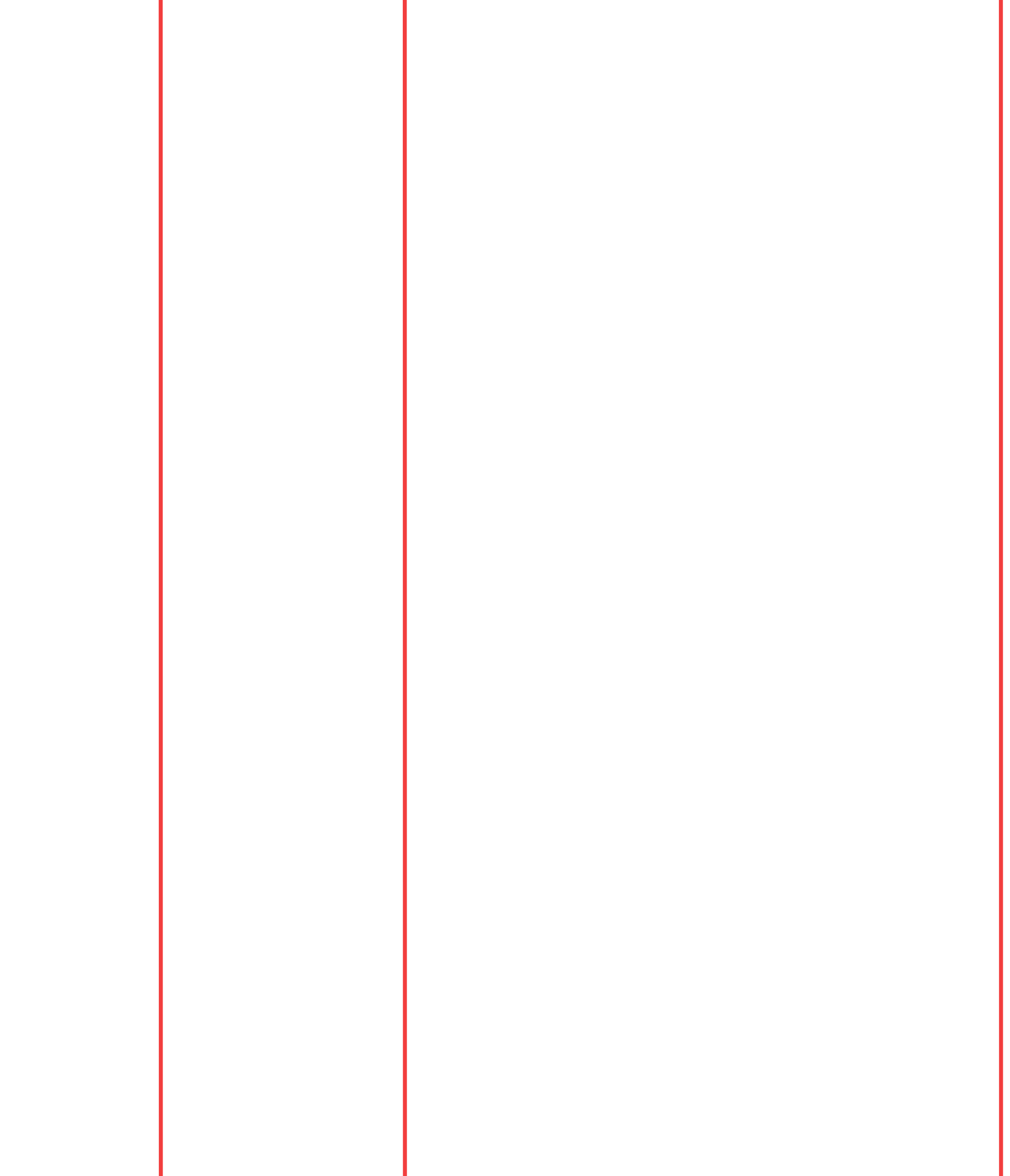
Architectural patterns provide reusable solutions for common system design problems, while anti-patterns describe common architectural mistakes to avoid.

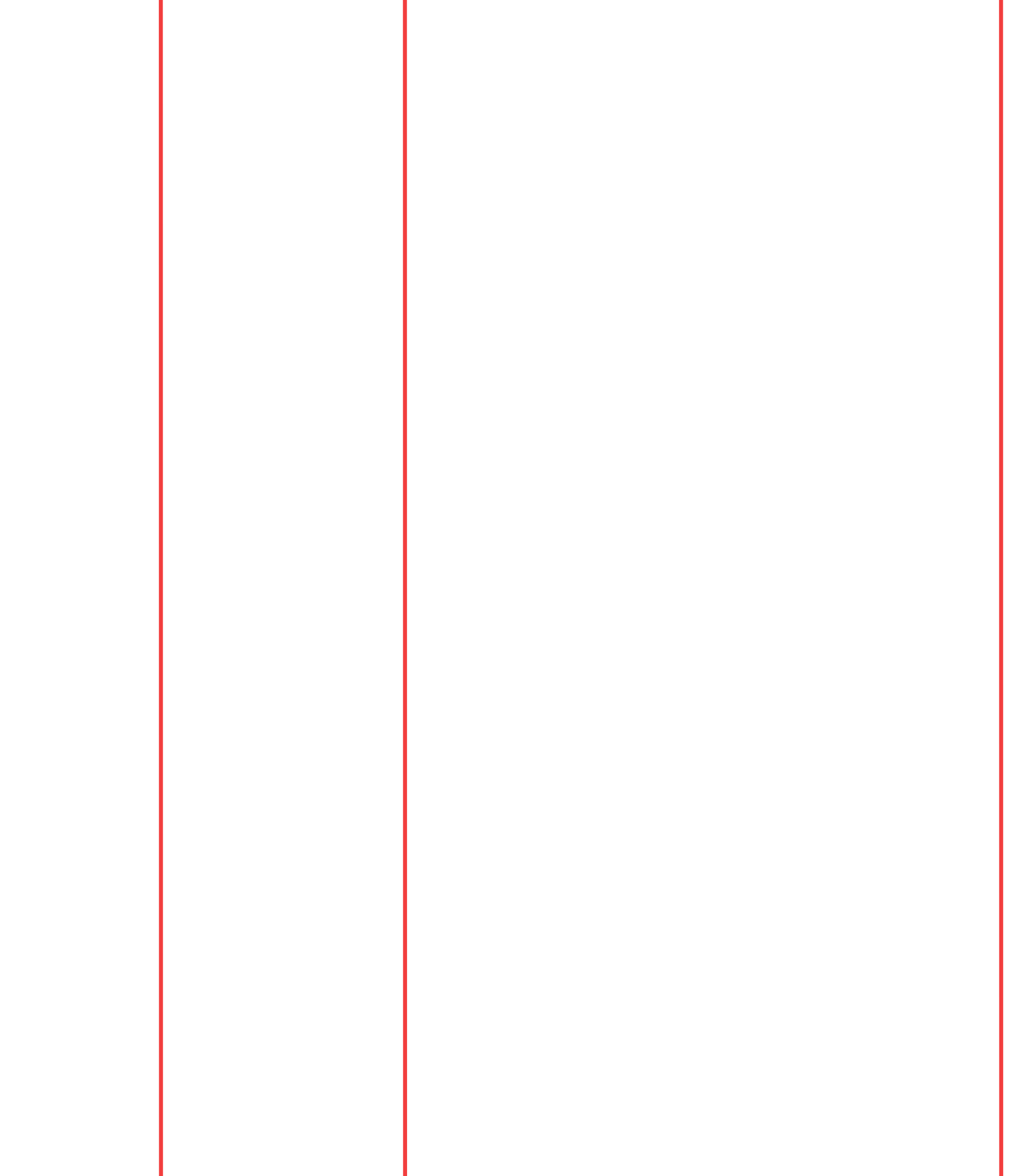
Identification Questions	
Key idea	Typical example
single large application	traditional enterprise apps
system divided into layers	web apps
clients require server services	web systems
data processed in stages	compilers
reusable components	enterprise software
systems react to events	messaging systems
decentralized nodes	Bitcoin

Services Questions	
	Answer
Is it a Service Architecture?	system composed of small independent services communicating through APIs
	bounded context and service independence
	scalability, independent deployment
Challenges	distributed complexity, debugging difficulty

- | Key idea |
|---|
| separate model, view, controller |
| services discover each other dynamically |
| publishers send events, subscribers receive them |
| aggregate requests to multiple services and aggregate results |
| route requests across servers |
| central controller manages workflow |
| services coordinate through events |
| distributed transactions with compensation |
| distributed big data processing |

(Very Common)	
Choreography	
decentralized	
event-driven	
high	
harder	





1

2

3